

Appendix

A. GPU Allocation Strategy

We build WoVR on top of RLinf [3] to support efficient distributed imagined rollouts and training. Concretely, we replace RLinf’s environment back-end with our learned world model, enabling scalable closed-loop rollouts without a ground-truth simulator. GPU allocation details are provided in Appendix.

The reinforcement-learning pipeline in RLinf can be decomposed into three components: *Generation*, *Simulator*, and *Training*. In WoVR, the *Simulator* is instantiated by the learned *world model*, which generates the next observation given the current observation and action. In the rollout phase, *Generation* performs policy inference to produce an (optionally chunked) action from the current observation, while the *Simulator* executes the action and returns the next observation; this closed-loop interaction repeats until a batch of trajectories is collected. In the optimization phase, *Training* updates the VLA policy using the collected trajectories, after which the system alternates back to rollout for the next iteration.

Following the system abstraction in RLinf-VLA, WoVR adopts a collocated (shared) GPU allocation strategy, where the three RL pipeline components co-exist on the same set of GPUs, with the *Simulator* implemented as the world-model rollout module. Unlike physical simulators that require dedicated device-side state, WoVR’s simulator is a neural network; thus, offload/onload can be naturally realized by swapping only the *model parameters* between GPU and host memory, without migrating any external simulator state. In its original form, collocated execution relied on frequent GPU↔CPU offload/onload to keep only one component resident on GPUs at a time; however, in embodied settings the simulator and generator must interact iteratively, making per-interaction offload/onload prohibitively expensive. Therefore, we use the modified collocated strategy: offload/onload for *Generation* and *Simulator* happens only at the beginning and end of the rollout phase, avoiding repeated transfers during closed-loop imagined interaction, as illustrated in Fig. 1.

B. Reward Modeling

B.1. Sparse Reward Model

For sparse supervision, we use a learned reward classifier that predicts task success from the predicted next observation. Concretely, given the generated observation $\tilde{o}_{t+1}$ from the world model, the sparse reward model R_{ψ}^{sp} estimates the probability of task success, and the binary reward is defined as

$$r_{t+1}^{\text{sp}} = \mathbb{I}\left(R_{\psi}^{\text{sp}}(\tilde{o}_{t+1}) \geq 0.5\right), \quad (1)$$

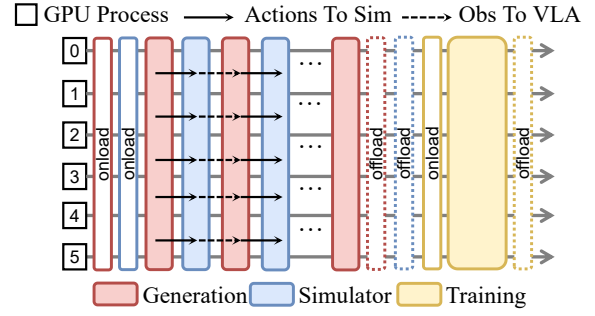


Figure 1. Collocated GPU Allocation Strategy

where $\mathbb{I}(\cdot)$ denotes the indicator function. Following HiL-SERL [2], the sparse reward classifier is implemented as a lightweight network and trained with binary cross-entropy loss on labeled success states, defined as

$$\mathcal{L}_{\text{binary}}(\psi) = -\frac{1}{N} \sum_{i=1}^N \left[r_i \log C_{\psi}(x_i) + (1 - r_i) \log (1 - C_{\psi}(x_i)) \right]. \quad (2)$$

B.2. Dense Reward Modeling

To provide finer-grained learning signals beyond binary success, we further introduce a dense reward model built on top of Qwen3-VL [1]. Instead of directly regressing a scalar reward, we formulate dense reward prediction as an ordinal visual progress estimation problem. Given four consecutive observations $\tilde{o}_{t-3:t} = [\tilde{o}_{t-3}, \tilde{o}_{t-2}, \tilde{o}_{t-1}, \tilde{o}_t]$ and the task description l , the dense reward model R_{ψ}^{de} predicts a discrete progress level

$$\ell_t \in \{0, 1, \dots, 10\}, \quad (3)$$

where 0 denotes no progress and 10 denotes task completion. The dense reward is then obtained by normalization:

$$r_t^{\text{de}} = \frac{\ell_t}{10}. \quad (4)$$

The model takes as input four consecutive RGB frames together with the task text, and outputs logits over 11 reward levels. Architecturally, we adopt a Qwen3-VL backbone with LoRA adaptation and attach a lightweight MLP reward head for classification. During training, labels are constructed automatically from trajectory rewards: failed trajectories are assigned level 0, successful states are assigned level 10, and pre-success states are linearly mapped to intermediate levels 1–9 according to their temporal proximity to the first successful state. To mitigate class imbalance, we further rebalance samples across reward levels during dataset construction.

The dense reward model is trained with a standard cross-entropy objective:

$$\mathcal{L}_{\text{dense}}(\psi) = \mathbb{E}_{(\tilde{o}_{t-3:t}, d, \ell_t)} [-\log p_{\psi}^{\text{de}}(\ell_t | \tilde{o}_{t-3:t}, d)]. \quad (5)$$

We provide an analysis of the trade-off between sparse and dense reward modeling in Sec. C.3.

C. Ablation Study

C.1. Ablation on World Model Mechanisms

We first conduct ablation studies on the core design choices of the proposed world model, aiming to understand how different context modeling mechanisms affect long-horizon video generation stability. Specifically, we investigate the following factors: (i) the number of memory frames used as visual context, (ii) the use of a fixed reference frame, and (iii) the effect of adding noise to context frames during training.

Experimental Variants. We compare the full WoVR model against three ablated variants:

- WoVR w/o ref, which removes the fixed reference frame from the context window;
- WoVR w. mem=1, which uses only a single-frame context;
- WoVR w/o noisy context, which disables noise injection on context frames during training.

All variants are trained and evaluated on the **LIBERO-Spatial** suite only. We train the world model using 1,500 VLA rollout trajectories and evaluate on a held-out set of 24 trajectories.

Quantitative Results. Table 1 reports the quantitative results measured by LPIPS, FID, and FVD under different rollout horizons. Compared to using a single-frame context, employing a multi-frame context with a fixed reference anchor significantly improves performance across all metrics.

To better understand the failure modes behind these quantitative trends, we provide qualitative comparisons in Fig. 2. As shown in the figure, models without a fixed reference frame or noisy context exhibit noticeable spatial drift and object disappearance over long-horizon rollouts, whereas the full WoVR model remains visually stable and consistent with the ground truth.

Removing the reference frame leads to a clear degradation in performance, especially under longer rollout horizons. This result suggests that anchoring the context with a fixed reference frame effectively suppresses error accumulation in the autoregressive feedback loop, which is critical for maintaining stability in long-horizon video generation.

Furthermore, disabling noise injection on context frames also results in noticeable performance drops. While the

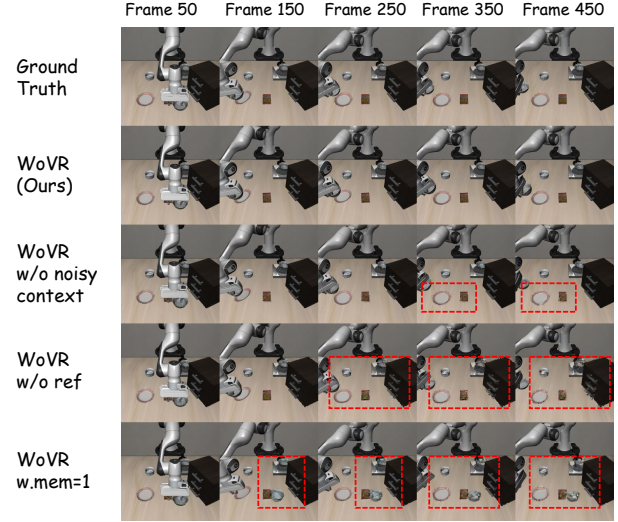


Figure 2. **Qualitative ablation results on LIBERO-Spatial.** Ablated variants exhibit error accumulation and visual drift under long-horizon rollouts, while the full WoVR model remains stable and consistent with the ground truth.

degradation is moderate for short rollouts, the gap becomes more pronounced as the rollout length increases. This observation indicates that adding mild noise to context frames improves robustness in long-horizon generation by reducing over-reliance on precise conditioning inputs, thereby alleviating the train–inference gap.

Overall, these results demonstrate that the proposed context modeling strategy—combining a fixed reference frame, a multi-frame memory window, and noisy context augmentation—plays a crucial role in stabilizing long-horizon video generation. Together, these mechanisms enable WoVR to maintain high fidelity and temporal consistency under closed-loop autoregressive inference, providing a more reliable simulator for downstream reinforcement learning.

C.2. Ablation on Policy Optimization Mechanisms

We next ablate key components in the policy optimization pipeline of WoVR, aiming to understand how different design choices affect downstream VLA task performance. In particular, we focus on mechanisms that facilitate stable policy learning and effective utilization of the learned world model.

Experimental Setup. All experiments are conducted on the **LIBERO-Spatial** suite, with the same training protocol, data budget, and evaluation procedure as described in Sec. 4.2. Specifically, the base VLA policy is pre-trained following the same demonstration setup as in Sec. 4.2, and policy optimization is performed using world-model-based

Table 1. Ablation study on world model mechanisms (LIBERO-Spatial). Rollout denotes the rollout horizon length.

Method	Rollout	Metrics			
		LPIPS↓	FID↓	FVD↓	FloLPIPS↓
WoVR (Ours)	512	0.091	36.687	73.493	0.154
	256	0.069	27.238	63.948	0.110
	128	0.051	20.780	49.017	0.081
WoVR w/o ref	512	0.133	73.942	123.502	0.168
	256	0.089	49.406	86.000	0.116
	128	0.064	35.559	86.146	0.090
WoVR w. mem=1	512	0.120	64.501	86.042	0.165
	256	0.086	46.790	81.742	0.117
	128	0.065	36.047	79.605	0.095
WoVR w/o noisy context	512	0.099	44.712	77.284	0.160
	256	0.074	31.691	61.660	0.115
	128	0.054	23.444	58.836	0.085

reinforcement learning. Task performance is measured by the average success rate over the LIBERO-Spatial tasks.

We compare the full WoVR framework against the following ablated variants:

- WoVR w/o KIR, which removes keyframe-based initialization and starts policy optimization from randomly sampled initial states in the world model;
- WoVR w/o PACE, which disables the co-evolution of the world model with the updated policy and keeps the world model fixed during policy optimization.

Quantitative Results. Table 2 reports the success rates on the LIBERO-Spatial suite. The full WoVR framework achieves the highest performance, with an average success rate of 0.815. Removing keyframe-based initialization leads to a noticeable drop in performance, reducing the success rate to 0.782. This result indicates that KIR plays an important role in stabilizing early-stage policy learning by providing meaningful initial states.

Disabling the co-evolution of the world model further degrades performance to 0.71. This suggests that continuously refining the world model with updated policy rollouts is critical for maintaining simulator accuracy and preventing compounding model errors during policy optimization.

C.3. Ablation on Reward Modeling

We further analyze the impact of different reward modeling choices on policy optimization, comparing a lightweight ResNet-based sparse reward model with a Qwen3-VL-based dense reward model.

Table 2. Ablation on policy optimization mechanisms on LIBERO-Spatial. Success rate is averaged over all tasks in the suite.

Method	Success Rate ↑
WoVR (Ours)	0.815
WoVR w/o KIR	0.782
WoVR w/o PACE	0.710

Experimental Setup. We keep all other components fixed and vary only the reward model used during policy optimization. The sparse reward model predicts binary task success, while the dense reward model provides fine-grained progress signals. Both variants are evaluated on the same training setup and metrics as in Sec. 4.2.

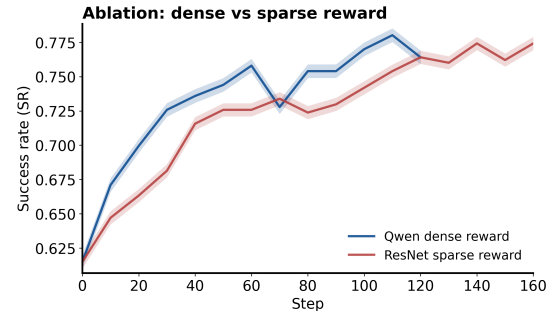


Figure 3. Ablation on reward modeling: dense vs. sparse reward.

Results and Analysis. Fig. 3 shows the learning curves under the two reward settings. We observe that the dense reward model improves sample efficiency in the early stage of training, leading to faster initial performance gains. However, both reward designs converge to similar final performance.

Despite its advantage in early learning, the dense reward model incurs significantly higher computational cost, as it relies on a large vision-language backbone (Qwen3-VL 2B). In practice, we find that rollout with the dense reward model is approximately $3\times$ slower than with the lightweight sparse reward model.

What’s more, binary rewards are widely adopted in manipulation benchmarks such as LIBERO as well as real-world robotic setups. Taking the above accounts into considerations, we adopt the sparse reward model in all main experiments.

References

- [1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025. 1
- [2] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105): eads5033, 2025. 1
- [3] Chao Yu, Yuanqing Wang, Zhen Guo, Hao Lin, Si Xu, Hongzhi Zang, Quanlu Zhang, Yongji Wu, Chunyang Zhu, Junhao Hu, et al. Rlinf: Flexible and efficient large-scale reinforcement learning via macro-to-micro flow transformation. *arXiv preprint arXiv:2509.15965*, 2025. 1